AZHL

Provider Earnings, Transactions & Withdrawal Backend Specification

Backend handoff document - API contracts, financial rules, database model, admin workflow and concurrency safeguards.

Key Business Example

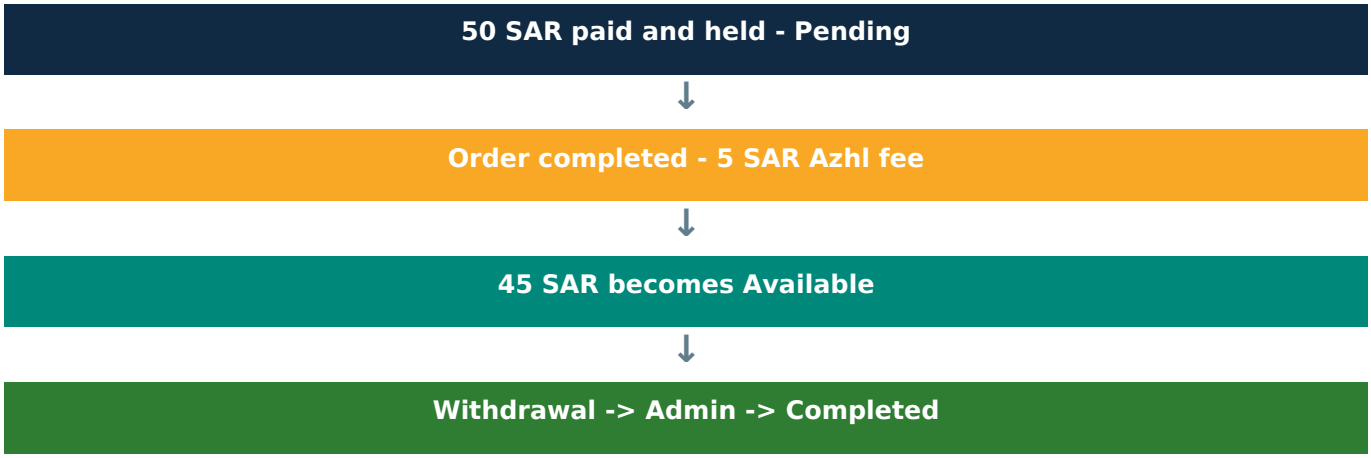
Customer order: 50 SAR

Azhl platform fee on completion: 5 SAR

Provider net credit: 45 SAR

Before completion, the full 50 SAR remains pending and cannot be withdrawn.

TOTAL EARNINGS	AVAILABLE	PENDING	WITHDRAWN
2,450 SAR	850 SAR	300 SAR	1,600 SAR



Prepared for Backend Team

Version: 1.0 | Currency: SAR | Scope: Provider + Marketplace compatible

Implementation Map

A fast reference for backend developers, API developers and admin-panel implementation.

#	Module	Primary Responsibility
01	Financial Model	Pending, available, earnings and withdrawn balances
02	Order Settlement	Deduct Azhl fee and create provider credit
03	Wallet Summary API	Return dashboard-level aggregates
04	Transaction API	Paginated Credit/Withdraw history with nullable filter
05	Withdrawal Request	Reserve funds and create request
06	Admin Processing	Accept, complete or reject with reason
07	Database Model	Wallets, transactions and withdrawals
08	Safety Rules	Locking, idempotency and invalid state protection
09	Marketplace Reuse	Reuse wallet engine for marketplace accounts

Non-negotiable rules

Never credit a completed order twice.

Never allow a pending amount to be withdrawn.

Reserve withdrawal funds immediately when a request is created.

Only a completed withdrawal increases total_withdrawn.

Admin rejection must contain a reason and release the reserved amount.

1. Financial Model

Each provider has one financial wallet view. The wallet values must remain consistent with order settlement and withdrawal records.

TOTAL EARNINGS	AVAILABLE	PENDING	WITHDRAWN
2,450 SAR	850 SAR	300 SAR	1,600 SAR

Metric	Definition	When it changes
Total Earnings	Sum of provider net earnings from completed orders.	Increases after an order is financially settled.
Pending Amount	Gross value of active/held provider orders.	Increases on hire/payment; decreases on completed settlement.
Available for Withdrawal	Net provider funds eligible for withdrawal.	Increases after completion; decreases when a withdrawal is requested.
Total Withdrawn	Funds successfully transferred to provider.	Increases only when admin marks withdrawal completed.

50 SAR example

Order payment received: 50 SAR

↓

Pending wallet balance: +50 SAR

↓

Order completed and confirmed

↓

Azhl fee deducted: 5 SAR

↓

Provider available balance: +45 SAR

Important

The fee is deducted from every completed provider order.

Store gross amount, fee amount and net amount separately for audit and reporting.

The gross 50 SAR should not be counted as provider total earnings; the provider earning is 45 SAR.

2. Order Settlement & Azhl Fee

Settlement formula

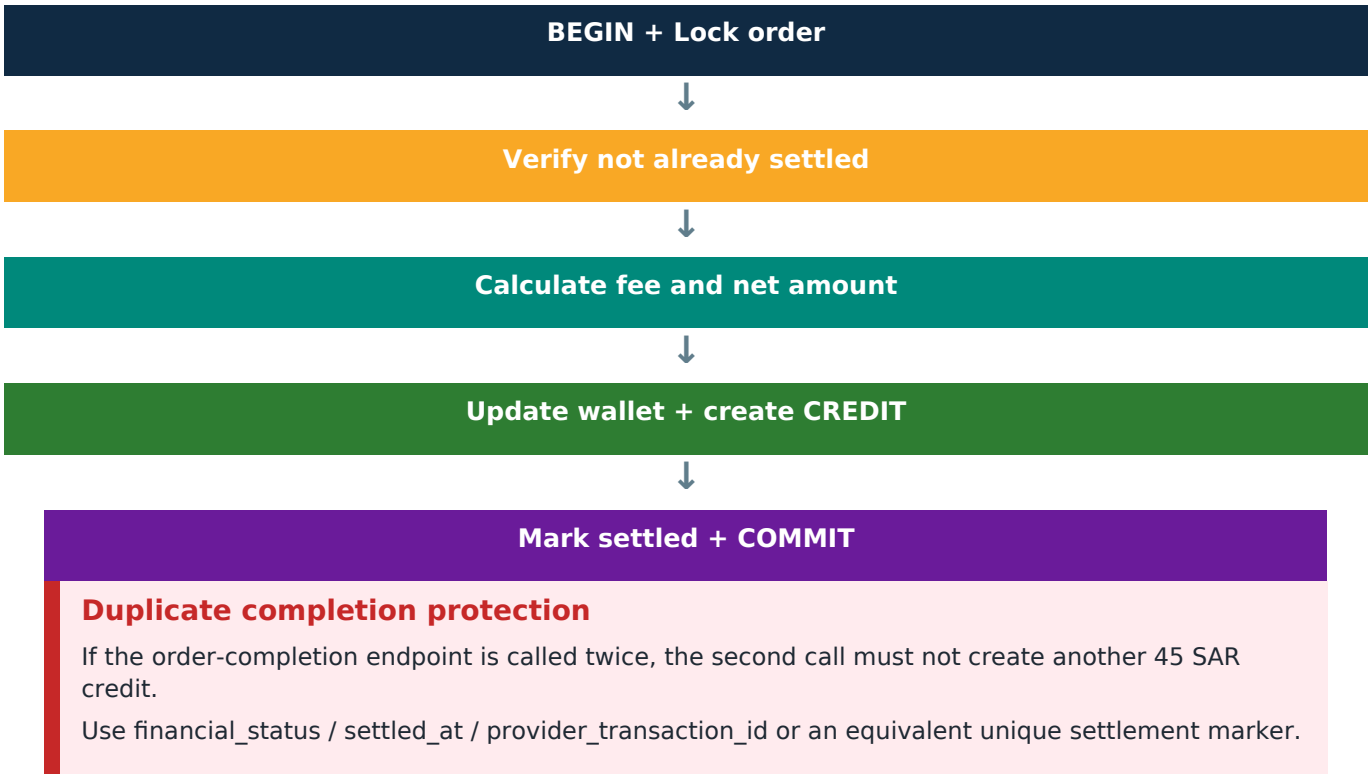
```
provider_net_amount = gross_order_amount - azhl_fee

Example:
50.00 SAR - 5.00 SAR = 45.00 SAR
```

Settlement effects

Operation	Wallet / Ledger Effect
Remove active order hold	pending_amount -= 50.00
Record Azhl revenue	platform_fee = 5.00
Credit provider	available_balance += 45.00
Increase provider earnings	total_earnings += 45.00
Create ledger row	wallet_transaction.type = credit
Mark order settlement	financial_status = settled

Recommended atomic flow



Duplicate completion protection

If the order-completion endpoint is called twice, the second call must not create another 45 SAR credit.

Use financial_status / settled_at / provider_transaction_id or an equivalent unique settlement marker.

3. Provider Wallet Summary API

```
GET /api/v1/provider/wallet/summary
Authorization: Bearer {token}
```

Success response

```
{
  "success": true,
  "message": "Provider wallet summary retrieved successfully.",
  "data": {
    "total_earnings": 2450.00,
    "pending_amount": 300.00,
    "available_for_withdrawal": 850.00,
    "total_withdrawn": 1600.00,
    "currency": "SAR"
  }
}
```

Calculation rules

- total_earnings = sum of net provider credits from completed orders.
- pending_amount = gross amount currently held for active/incomplete orders.
- available_for_withdrawal = settled net credits not withdrawn and not reserved in active withdrawal requests.
- total_withdrawn = sum of withdrawals where status = completed only.

> **Do not count**
>
> Requested withdrawal -> NOT total withdrawn.
>
> Accepted withdrawal -> NOT total withdrawn.
>
> Rejected withdrawal -> NOT total withdrawn.

4. Transaction History API

The transaction endpoint returns a single combined timeline containing provider credits and provider withdrawals. Newest transactions should appear first.

```
GET /api/v1/provider/wallet/transactions?page=1&per_page=20&filter=credit
Authorization: Bearer {token}
```

Nullable filter

filter	Result
null	Credit + Withdraw
all	Credit + Withdraw
credit	Only order earning credits
withdraw	Only withdrawal request/history records

Pagination contract

Parameter	Rule
page	Default 1
per_page	Default 20
maximum per_page	Recommended 100
sort	created_at DESC

Every transaction must expose a label

```
Credit transaction:
{
  "type": "credit",
  "label": "Credit"
}

Withdrawal transaction:
{
  "type": "withdraw",
  "label": "Withdraw"
}
```

5. Transaction API Response Contract


```
{
  "success": true,
  "message": "Transactions retrieved successfully.",
  "data": {
    "transactions": [
      {
        "id": 125,
        "type": "credit",
        "label": "Credit",
        "amount": 45.00,
        "currency": "SAR",
        "created_at": "2026-08-19T10:30:00+05:00",
        "credit": {
          "order_id": 812,
          "order_no": "ORD-000812",
          "order_title": "AC Repair Service",
          "order_status": "completed",
          "gross_amount": 50.00,
          "azhl_fee": 5.00,
          "net_amount": 45.00,
          "completed_at": "2026-08-19T10:30:00+05:00"
        },
        "withdraw": null
      },
      {
        "id": 124,
        "type": "withdraw",
        "label": "Withdraw",
        "amount": 200.00,
        "currency": "SAR",
        "created_at": "2026-08-18T14:15:00+05:00",
        "credit": null,
        "withdraw": {
          "withdrawal_id": 55,
          "withdrawal_no": "WDR-000055",
          "bank_name": "Al Rajhi Bank",
          "status": "requested",
          "requested_at": "2026-08-18T14:15:00+05:00",
          "completed_at": null,
          "rejected_at": null,
          "rejection_reason": null
        }
      }
    ],
    "pagination": {
      "current_page": 1,
      "per_page": 20,
      "last_page": 6,
      "total": 117,
      "from": 1,
      "to": 20,
      "has_more": true
    }
  }
}
```

6. Provider Withdrawal Request

```
POST /api/v1/provider/withdrawals
Authorization: Bearer {token}
```

Request body

```
{
  "amount": 200.00,
  "bank_account_id": 12
}
```

Validation

- amount is required, numeric and greater than 0.
- bank_account_id is required.
- Bank account must belong to the authenticated provider.
- Requested amount cannot exceed available balance.
- Funds must be reserved atomically before request creation completes.

Successful response

```
{
  "success": true,
  "message": "Withdrawal request submitted successfully.",
  "data": {
    "id": 56,
    "withdrawal_no": "WDR-000056",
    "amount": 200.00,
    "currency": "SAR",
    "status": "requested",
    "bank_account": {
      "id": 12,
      "bank_name": "Al Rajhi Bank",
      "iban": "SA****************1234"
    },
    "requested_at": "2026-08-19T11:20:00+05:00"
  }
}
```

Insufficient balance error

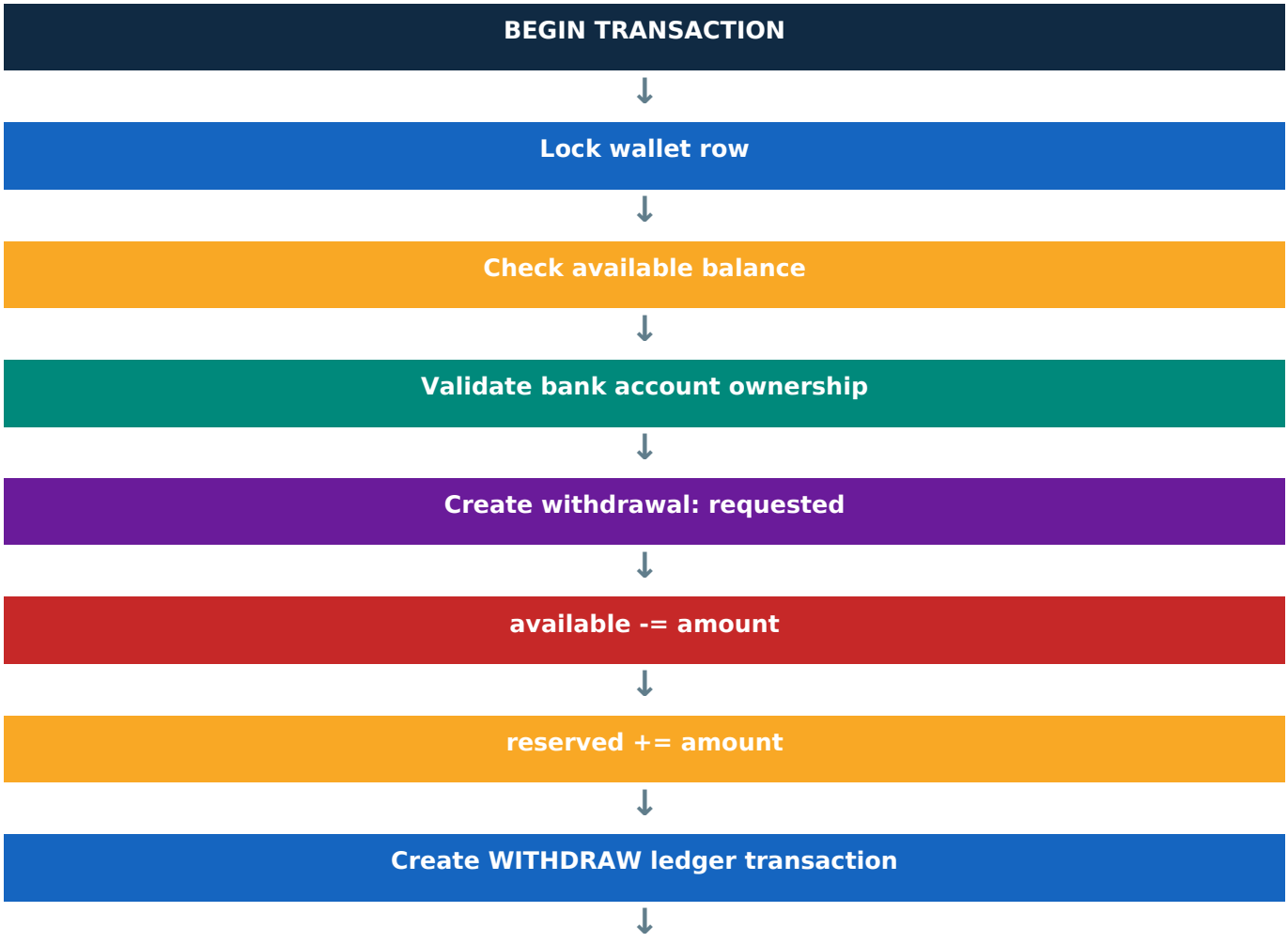
```
HTTP 422
{
  "success": false,
  "message": "Insufficient available balance.",
  "errors": {
    "amount": [
      "The withdrawal amount cannot exceed your available balance."
    ]
  }
}
```

7. Withdrawal Balance Reservation

A withdrawal request must immediately reduce spendable availability, even before admin transfers the money. This prevents duplicate withdrawal requests against the same funds.

Stage	Available	Reserved	Total Withdrawn
Before request	500 SAR	0 SAR	1,000 SAR
Request 400 SAR	100 SAR	400 SAR	1,000 SAR
If completed	100 SAR	0 SAR	1,400 SAR
If rejected	500 SAR	0 SAR	1,000 SAR

Atomic request flow



Concurrency safeguard

If two withdrawal requests arrive together, lock the wallet row so the same available balance cannot be reserved twice.

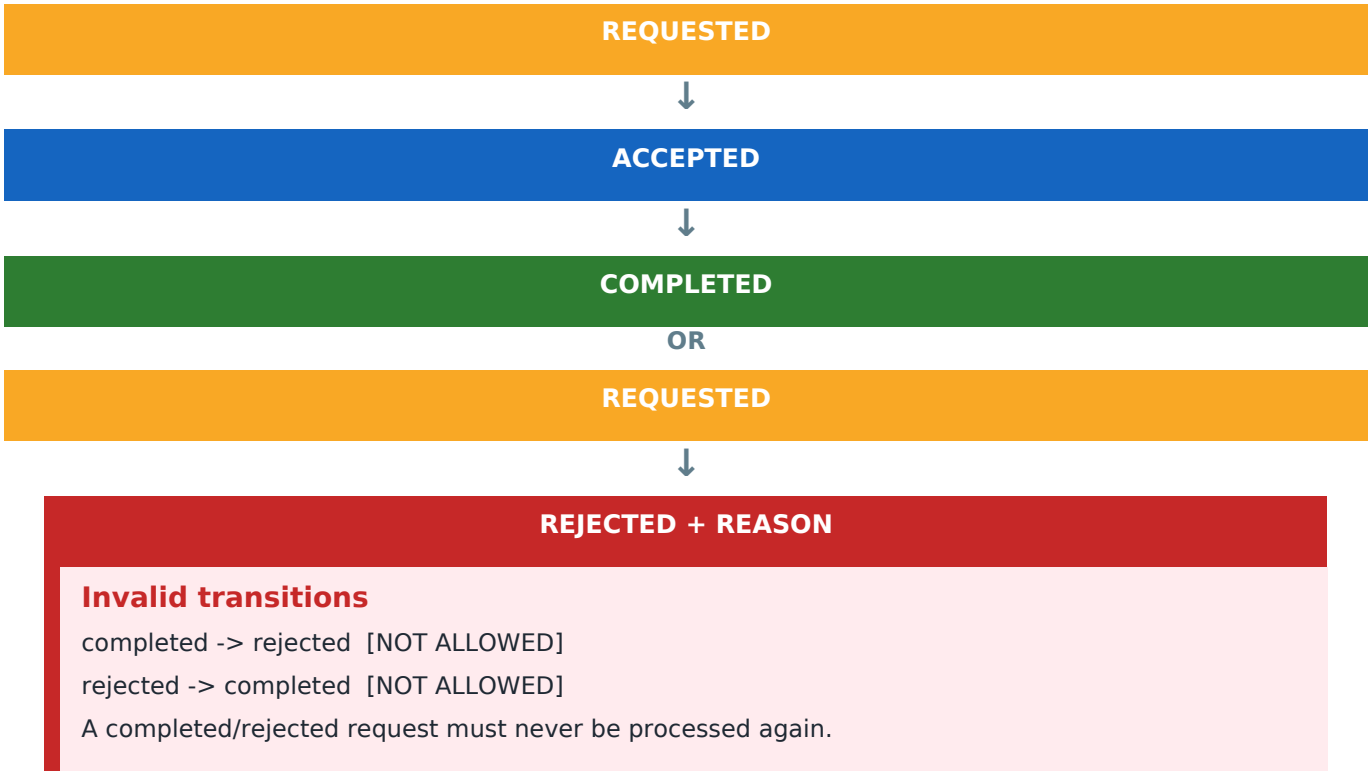
8. Admin Withdrawal Management

```
GET /api/v1/admin/withdrawals?status=requested&page=1
```

Admin list fields

Field	Purpose
withdrawal_no	Human-readable request reference
account_type	provider or marketplace
account_id / name	Owner of wallet
amount / currency	Requested transfer value
status	requested / accepted / completed / rejected
available_balance	Operational context for admin
bank details	Bank, account holder and IBAN
requested_at	Request timestamp

Valid status lifecycle



Invalid transitions

completed -> rejected [NOT ALLOWED]

rejected -> completed [NOT ALLOWED]

A completed/rejected request must never be processed again.

9. Admin Action APIs

Accept

```
PATCH /api/v1/admin/withdrawals/{id}/accept

{
  "success": true,
  "message": "Withdrawal request accepted successfully.",
  "data": {
    "id": 56,
    "withdrawal_no": "WDR-000056",
    "status": "accepted",
    "accepted_at": "2026-08-19T12:00:00+05:00"
  }
}
```

Complete after bank transfer

```
PATCH /api/v1/admin/withdrawals/{id}/complete

Request:
{
  "bank_reference": "TXN-984512784"
}

Result: reserved_balance decreases and total_withdrawn increases.
```

Reject with reason

```
PATCH /api/v1/admin/withdrawals/{id}/reject

{
  "reason": "The submitted IBAN could not be verified."
}
```

> **Reject balance behavior**
>
> reserved_balance -= withdrawal amount
>
> available_balance += withdrawal amount
>
> status = rejected
>
> rejection_reason and rejected_at must be stored.

10. Recommended Database Model

wallets

Field	Notes
id	Primary key
walletable_type / walletable_id	Polymorphic owner: Provider or Marketplace
total_earnings	Net credited earnings
pending_amount	Gross held orders
available_balance	Spendable/withdrawable balance
reserved_balance	Locked active withdrawal funds
total_withdrawn	Successfully paid out amount
currency	SAR
timestamps	created_at / updated_at

wallet_transactions

Field	Example / Meaning
type	credit | withdraw
reference_type	order | withdrawal
reference_id	812 | 56
gross_amount / fee_amount / net_amount	50.00 / 5.00 / 45.00 for a provider order credit
amount / currency	Ledger amount + SAR
description	Optional human-readable description

withdrawals

Field	Notes
withdrawal_no	WDR-000056
wallet_id	Owner wallet
bank_account_id	Selected transfer account
amount / currency / status	Requested amount + SAR + lifecycle status
rejection_reason / bank_reference	Reject reason or transfer reference
lifecycle + admin audit	requested/accepted/completed/rejected timestamps + acting admin IDs

11. Common API Response Standards

Success

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {}
}
```

Validation

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "amount": ["The amount field is required."]
  }
}
```

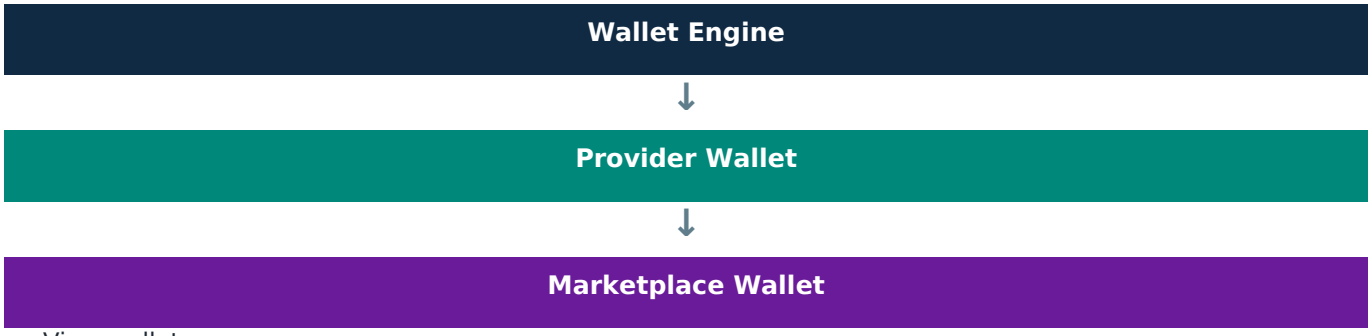
HTTP status guidance

Code	Use
200	Successful GET / update
201	Resource created
401	Unauthenticated
403	Forbidden
404	Not found
422	Validation / business rule failure
500	Unexpected server failure

> **Amount formatting**
>
> Return numeric decimal values consistently, e.g. 45.00 with currency = SAR.
>
> Avoid mixing 45, "45 SAR" and 45.0 across endpoints.

12. Marketplace Compatibility

The wallet engine should be reusable for Provider and Marketplace accounts. The business rules can remain the same while fee behavior may be configured per account/order type.



- View wallet summary.
- View paginated transaction history.
- Save/select bank account.
- Submit withdrawal request.
- Track requested / accepted / completed / rejected status.
- Admin must receive account_type to distinguish provider vs marketplace.

```
{
  "account_type": "provider"
}

or

{
  "account_type": "marketplace"
}
```

13. Backend Implementation Checklist

✓	Provider hire/payment increases pending_amount by gross order amount.
✓	Pending amount cannot be withdrawn.
✓	Order completion deducts Azhl fee once.
✓	50 SAR example settles to 5 SAR Azhl + 45 SAR provider.
✓	Credit transaction stores gross, fee and net amounts.
✓	Transaction endpoint supports pagination.
✓	Transaction filter is nullable and supports all / credit / withdraw.
✓	Every transaction returns type and label.
✓	Withdrawal request validates bank ownership and balance.
✓	Requested amount is reserved immediately.
✓	Rejected withdrawal returns reserved funds to available balance.
✓	Completed withdrawal increases total_withdrawn.
✓	Admin rejection requires reason.
✓	Completed/rejected withdrawals cannot be processed again.
✓	Order settlement is idempotent.
✓	Wallet mutation uses DB transaction and row locking.
✓	Same wallet architecture can serve Provider + Marketplace.

14. Endpoint Summary

Method	Endpoint	Purpose
GET	/api/v1/provider/wallet	Dashboard summary + recent items (optional combined endpoint)
GET	/api/v1/provider/wallet/summary	Provider financial summary
GET	/api/v1/provider/wallet/transactions	Paginated/filterable combined transaction history
POST	/api/v1/provider/withdrawals	Create withdrawal request
GET	/api/v1/provider/withdrawals	Provider withdrawal list
GET	/api/v1/admin/withdrawals	Admin request panel
PATCH	/api/v1/admin/withdrawals/{id}/accept	Accept request
PATCH	/api/v1/admin/withdrawals/{id}/complete	Mark bank transfer completed
PATCH	/api/v1/admin/withdrawals/{id}/reject	Reject with reason

Transaction query example

```
GET /api/v1/provider/wallet/transactions?page=1&per_page=20&filter=withdraw
```

> **Final expected financial flow**
>
> Customer pays 50 SAR -> 50 SAR pending.
>
> Order completes -> 5 SAR Azhl fee + 45 SAR provider available.
>
> Provider requests withdrawal -> amount moves from available to reserved.
>
> Admin completes bank transfer -> reserved clears and total_withdrawn increases.
>
> If admin rejects -> amount returns to available and rejection reason is visible in transaction history.